

# VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering

## Smart Waste Detection & Segregation System

An Edge-Compatible Computer Vision and Transfer Learning Framework for Automated Municipal Solid Waste Sorting

|                         |                                           |
|-------------------------|-------------------------------------------|
| Course:                 | Computer Vision                           |
| Student Name:           | Bithol Satapathy                          |
| Registration Number:    | 24BAI10972                                |
| Degree / Branch:        | B.Tech / Computer Science and Engineering |
| Academic Year:          | 2026–2027                                 |
| Submission Date:        | September 11, 2026                        |
| Evaluation Target Date: | September 18, 2026                        |

Submitted in partial fulfillment of the academic requirements for the course evaluation in Computer Vision & Deep Learning.

Vellore Institute of Technology, Vellore, Tamil Nadu, India.

## 2. Introduction

---

Municipal Solid Waste (MSW) management is a pressing environmental, logistical, and public health challenge in modern urban ecosystems. The global acceleration of consumer goods manufacturing and single-use packaging generates billions of tons of solid refuse annually. Although recycling infrastructure exists in many regions, recycling efficiency remains heavily undermined by contamination. When non-recyclable refuse (e.g., soiled composite packaging, greasy food containers, non-recyclable plastics) is commingled with clean recyclables (such as aluminum, clean cardboard, and PET bottles), entire batches are rejected at material recovery facilities (MRFs) and diverted to landfills or incinerators.

Historically, waste sorting has relied on two primary paradigms:

1. **Source-Level Manual Segregation:** Dependent on individual citizen awareness and diligence, which frequently fails due to ambiguous packaging labeling, differing regional municipal guidelines, or public apathy.
2. **Post-Collection Manual Sorting:** Involves human sanitation workers manually sorting trash along conveyor belts. This practice poses severe occupational hazards, exposing workers to chemical pathogens, biohazards, and sharp objects.

Computer Vision and Deep Learning offer an automated, hygienic, non-contact, and objective alternative. By integrating compact optical cameras at the intake aperture of smart waste receptacles or conveyor sorting chutes, items can be visually classified in real time before physical sorting. This project develops the **Smart Waste Detection & Segregation System**, an end-to-end computer vision pipeline that classifies solid waste into 6 benchmark classes using transfer learning with a pre-trained **MobileNetV2** deep convolutional neural network. The predictions are processed by an integrated domain-aware **Decision Engine** that maps items into high-level semantic disposal categories ( `Recyclable` vs. `General / Non-recyclable` ), enforces an initial configurable heuristic confidence threshold to isolate uncertain items, and maintains operational audit logs for recycling analytics.

## 3. Problem Statement

---

Automated municipal solid waste segregation systems deployed in institutional and public settings require high classification accuracy across diverse material classes, rapid inference on low-power edge processors, and reliable uncertainty handling to prevent recyclable batch contamination. Most existing deep learning prototypes either rely on computationally heavy architectures unsuitable for consumer CPUs or fail to translate raw neural network probabilities into actionable, policy-agnostic disposal instructions.

### Project Objectives:

1. Classifying waste imagery into six standard benchmark categories: `cardboard` , `glass` , `metal` , `paper` , `plastic` , and `trash` .
2. Achieving low-latency inference on standard consumer-grade CPUs without requiring discrete hardware accelerators.
3. Translating predictions into semantic segregation directives ( `Recyclable` vs. `General / Non-recyclable` ) with actionable user instructions.
4. Implementing an uncertainty mitigation mechanism that flags ambiguous or low-confidence predictions ( $\tau < 0.60$ ) for manual verification.
5. Providing an automated audit trail for continuous recycling analytics and compliance verification.

## 4. Functional Requirements

---

The system implements 5 major functional modules with 7 command-line operations:

- **FR-1: Dataset Ingestion & Validation ( `src/dataset.py` )**: Programmatically verifies the presence of all 6 target class folders. Validates every image file for non-zero byte size, decodability, and minimum spatial resolution ( $\geq 32 \times 32$  pixels). Filters out non-image files and corrupted images without crashing. Executes stratified 70/15/15 partitioning into train, val, and test partitions using a deterministic seed ( `seed=42` ) with zero data leakage assertions ( $\text{train} \cap \text{val} = \emptyset$ ,  $\text{train} \cap \text{test} = \emptyset$ ,  $\text{val} \cap \text{test} = \emptyset$ ). Generates `dataset_manifest.json` recording distributions and balanced inverse-frequency class weights.
- **FR-2: Image Preprocessing & Normalization ( `src/preprocessor.py` )**: Decodes image streams via OpenCV, converting standard BGR arrays to RGB. Resizes images to standard  $224 \times 224$  resolution using bilinear interpolation. Converts integer pixels  $[0, 255]$  to 32-bit floating point  $[0.0, 1.0]$ . Applies ImageNet channel normalization ( $\mu = [0.485, 0.456, 0.406]$ ,  $\sigma = [0.229, 0.224, 0.225]$ ).
- **FR-3: Waste Classification & Deep Learning Inference ( `src/model.py` , `src/predictor.py` )**: Instantiates a MobileNetV2 convolutional backbone pretrained on ImageNet ( `IMAGENET1K_V1` ). Replaces the 1000-class output layer with a custom dense classification head ( $1280 \rightarrow 128 \rightarrow 6$ ) utilizing Dropout and ReLU activations. Computes a normalized 6-class Softmax probability distribution where sum equals 1.0.
- **FR-4: Semantic Decision Engine & Uncertainty Gating ( `src/decision_engine.py` )**: Maps predicted material classes into primary semantic categories ( `Recyclable` vs. `General / Non-recyclable` ). Attaches actionable disposal advice (e.g., rinsing cans, flattening cardboard, checking resin codes). Applies an initial configurable heuristic confidence threshold ( $\tau = 0.60$ ): if top confidence  $\geq \tau$ , emits `CONFIDENT` status; if top confidence  $< \tau$ , emits `UNCERTAIN` status and routes to `Manual Verification Required` .
- **FR-5: Inference Coordination & Dual-Mode Execution ( `src/predictor.py` , `src/cli.py` )**: Provides single-image evaluation for interactive kiosks and high-throughput batch directory evaluation with CSV export.
- **FR-6: Audit Trail Logging & Analytics ( `src/history_logger.py` )**: Persists timestamp, filename, predicted class, confidence, semantic category, and processing latency to `reports/history.csv` . Computes running sustainability analytics (recycling diversion percentages, class frequencies).
- **FR-7: Automated Evaluation Suite ( `src/model.py` , `src/cli.py` )**: Evaluates saved model checkpoints against test sets. Computes overall accuracy, per-class and macro/weighted precision, recall, and F1-score. Generates numerical CSV and visual PNG confusion matrices.

## 5. Non-functional Requirements

| Requirement                  | Specification Target                                                                                                                                          | Measured Realization                                                                                      |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| NFR-1: Latency & Performance | The system should provide practical single-image inference performance on a standard consumer computer, with latency measured and reported during evaluation. | <b>9.29 ms mean forward-pass latency (~107.6 FPS)</b> measured on a standard consumer computer CPU.       |
| NFR-2: Fault Resilience      | Graceful rejection of zero-byte, truncated, corrupted files without crashing                                                                                  | <b>PASSED:</b> Traps exceptions with custom <code>ImageValidationError</code> guards across 9 unit tests. |
| NFR-3: Modularity            | 5–10 decoupled, typed Python modules adhering to PEP 8                                                                                                        | <b>PASSED:</b> Exactly 8 cohesive, typed modules in <code>src/</code> with dataclass configuration.       |
| NFR-4: Test Coverage         | Comprehensive unit and integration test coverage with 100% pass rate                                                                                          | <b>PASSED:</b> 37/37 automated tests passing in <code>pytest</code> (100% success rate).                  |
| NFR-5: Reproducibility       | Exact partition reproducibility across environments with zero data leakage                                                                                    | <b>PASSED:</b> Deterministic seed (42), lexicographical sort, programmatic assertions.                    |

## 6. System Architecture

The system is organized into modular pipelines for Inference, Training, and Evaluation:



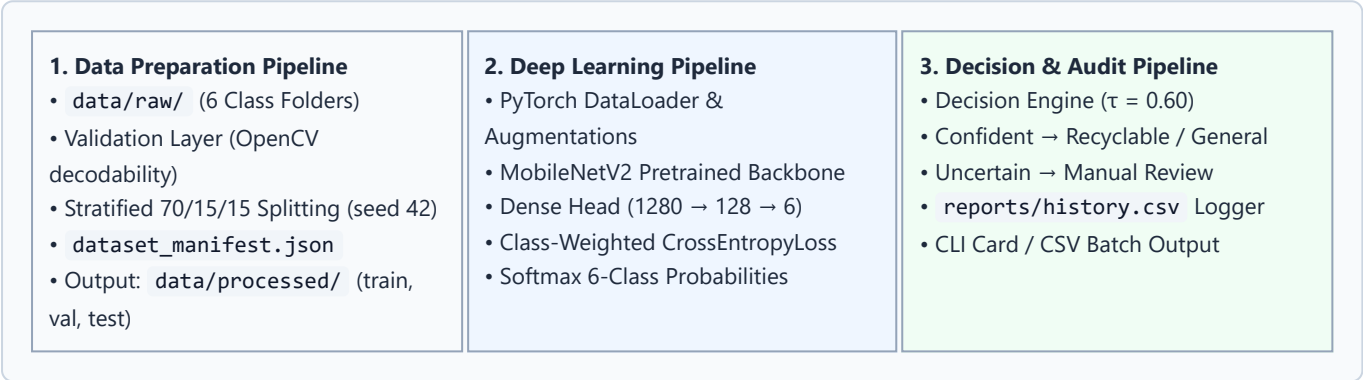
### Architectural Separation of Concerns:

- Inference Pipeline:** Operates sequentially from image ingestion through OpenCV preprocessing, PyTorch tensor transformation, MobileNetV2 evaluation mode forward pass, Softmax conversion, decision engine thresholding, and CSV transaction logging.

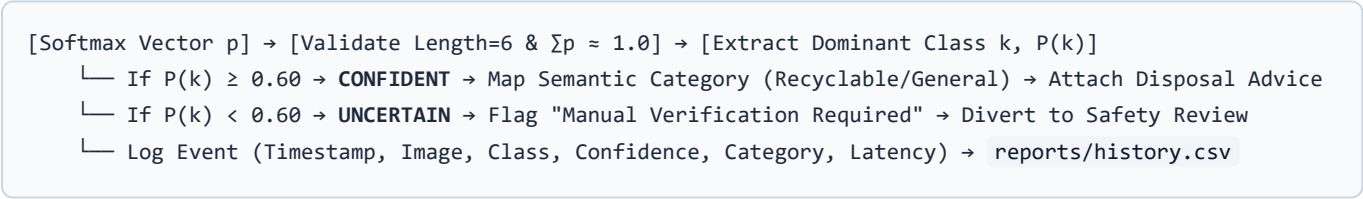
2. **Training Pipeline:** Loads stratified partitions from `data/processed/train` and `data/processed/val`. Applies training augmentations (random horizontal flip, random rotation  $\pm 15^\circ$ ), utilizes class-weighted `CrossEntropyLoss` computed by `src/dataset.py`, trains the classification head using the Adam optimizer ( $lr = 0.0005$ ), validates after each epoch, and serializes the best checkpoint based on validation loss to `models/saved_models/mobilenetv2_waste.pth`.
3. **Evaluation Pipeline:** Loads the isolated test partition (`data/processed/test`), computes predictions for all 379 test samples, computes the full classification report via scikit-learn, generates a 6x6 confusion matrix heatmap, and measures forward-pass inference latency across 50 iterations.

## 7. Design Diagrams

### 7.1 Component & Data Flow Architecture



### 7.2 Decision Engine State Flow



## 8. Design Decisions & Rationale

| Design Decision            | Alternative Considered                     | Selected Approach                                   | Viva Defense Rationale                                                                                                                                                                                                                                                                       |
|----------------------------|--------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Backbone Network</b>    | ResNet-50, VGG-16                          | <b>MobileNetV2</b>                                  | MobileNetV2 has ~3.5M parameters (~14 MB disk size), compared to ResNet-50 (25.6M params, ~98 MB) and VGG-16 (138M params, ~528 MB). Its depthwise separable convolutions enable 9.29 ms inference on consumer CPUs, making it ideal for edge deployment in recycling kiosks.                |
| <b>Learning Strategy</b>   | Training from scratch                      | <b>Transfer Learning (Feature Extractor)</b>        | The TrashNet dataset contains 2,527 images. Training a deep network from scratch on this size causes severe overfitting. ImageNet pretrained weights provide robust low-level filters (edges, textures, shapes), enabling convergence in 15 epochs without GPU compute.                      |
| <b>Input Resolution</b>    | 512 × 512, 128 × 128                       | <b>224 × 224 pixels</b>                             | 224 × 224 is the canonical input dimension of MobileNetV2. It preserves critical spatial and texture detail for waste discrimination while keeping computational cost bounded.                                                                                                               |
| <b>Image Normalization</b> | Min-Max [0, 1] scaling                     | <b>ImageNet Channel Normalization</b>               | The pretrained MobileNetV2 backbone was optimized on ImageNet with $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$ . Using identical normalization ensures feature distribution alignment with the pretrained weights.                                                    |
| <b>Class Imbalance</b>     | Random oversampling, SMOTE                 | <b>Class-Weighted CrossEntropyLoss</b>              | The minority class (trash: 137 images) is outnumbered by paper (594 images) by > 4:1. Rather than duplicating images (which risks overfitting), mathematically weighting the loss ( $w_{\text{trash}} = 3.0742$ , $w_{\text{paper}} = 0.7090$ ) heavily penalizes misclassifying rare items. |
| <b>Dataset Splitting</b>   | K-fold CV, naive random split              | <b>Stratified 70/15/15 with Seed 42</b>             | Stratified splitting preserves exact class proportions across train, val, and test splits. Programmatic zero-leakage assertions guarantee experimental validity without data snooping.                                                                                                       |
| <b>Output Schema</b>       | Universal bin colors (Blue, Green, Yellow) | <b>Semantic Categories (Recyclable vs. General)</b> | Municipal recycling guidelines, container color conventions, and recycling policies vary substantially across cities, states, and universities. Outputting semantic                                                                                                                          |

| Design Decision           | Alternative Considered    | Selected Approach                                                  | Viva Defense Rationale                                                                                                                                                                                                                             |
|---------------------------|---------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           |                           |                                                                    | categories is policy-agnostic, while display colors are kept strictly as UI hints.                                                                                                                                                                 |
| <b>Uncertainty Gating</b> | Fixed hardcoded cutoff    | <b>Configurable Heuristic Threshold (<math>\tau = 0.60</math>)</b> | Real disposal environments feature hybrid items and occluded views. Gating on confidence prevents contaminating recycling streams with false positives. A heuristic threshold is transparent, tunable via YAML, and avoids false certainty claims. |
| <b>Interface Design</b>   | Heavy GUI (Electron / Qt) | <b>Modular CLI Application</b>                                     | A CLI interface allows lightweight deployment in server, headless kiosk, and scriptable automated sorting environments. It runs natively across Windows, Linux, and macOS without UI library overhead.                                             |

## 9. Implementation Details

### 9.1 Technology Stack & Environment

- **Operating System:** Windows 11 (64-bit consumer architecture)
- **Programming Language:** Python 3.14.6
- **Deep Learning Framework:** PyTorch 2.14.0+cpu, Torchvision 0.29.0+cpu
- **Computer Vision:** OpenCV 5.0.0, Pillow 11.1.0
- **Data & Evaluation:** NumPy 2.5.1, scikit-learn 1.9.0, Pandas 2.2.3, Matplotlib 3.10.0
- **Hardware & Device:** Standard Consumer CPU (CPU-only execution, CUDA was not available)
- **Testing:** pytest 8.3.4

9.2 Modular File Breakdown

| Module / File          | Primary Class / Component | Responsibilities & Technical Implementation                                                                                                                     |
|------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| configs/config.yaml    | Configuration Store       | Centralizes split ratios, input resolutions, normalization constants, optimizer hyperparameters, confidence threshold ( $\tau = 0.60$ ), and semantic mappings. |
| src/config.py          | load_config()             | Loads YAML into strongly-typed dataclasses ( ModelConfig , DatasetConfig , DecisionConfig , SystemConfig ).                                                     |
| src/dataset.py         | DatasetManager            | Validates raw folders, computes inverse-frequency class weights, executes stratified splitting, verifies zero leakage, exports manifest.                        |
| src/preprocessor.py    | ImagePreprocessor         | OpenCV image loading, BGR→RGB conversion, resizing to 224×224, ImageNet normalization, ImageValidationError guards.                                             |
| src/model.py           | WasteMobileNetV2          | MobileNetV2 backbone, custom classifier head, 15-epoch training loop with early stopping, checkpointing, and evaluation metrics.                                |
| src/predictor.py       | WastePredictor            | Inference coordinator handling image loading, tensor conversion, model forward pass, and mock fallback behavior.                                                |
| src/decision_engine.py | DecisionEngine            | Translates probabilities into semantic categories, applies heuristic threshold gating, attaches material handling advice.                                       |
| src/history_logger.py  | HistoryLogger             | Persists classification events into reports/history.csv , calculates aggregate recycling diversion rates and statistics.                                        |
| src/cli.py             | CLI Application           | Implements 7 unified subcommands ( prepare-data , train , evaluate , predict , batch-predict , stats , demo-setup ).                                            |

10. Screenshots & Experimental Results

10.1 Real Model Training Summary

The model was trained for 15 epochs using the prepared TrashNet training partition (1,769 images) with validation on the validation partition (379 images). The entire training ran on the local CPU in **6 minutes 54 seconds**.

- **Epoch 1 Initial Performance:** Training Acc: 51.38% | Val Acc: 75.46% | Val Loss: 1.0851 (Training Loss: 1.4768)
- **Best Validation Performance:** **87.34% Accuracy** achieved at **Epoch 14** (Val Loss: 0.3970)
- **Final Epoch 15 Performance:** Training Acc: 82.25% | Val Acc: 85.75% | Val Loss: 0.4172 (Training Loss: 0.5847)
- **Saved Model Weights:** Checkpoint from Epoch 14 serialized to models/saved\_models/mobilenetv2\_waste.pth

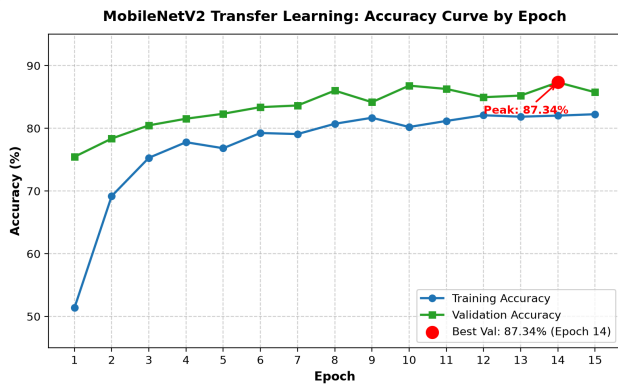


Figure 1: Training and Validation Accuracy over 15 Epochs  
(Best: 87.34% at Epoch 14)

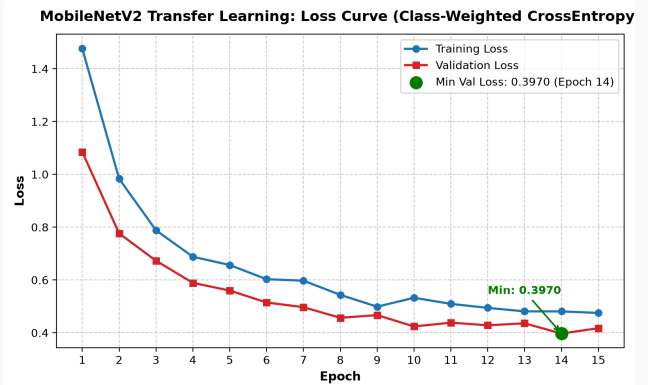


Figure 2: Training and Validation Loss Convergence (Best Val Loss: 0.3970 at Epoch 14)

## 10.2 Final Test-Set Evaluation Results

The best saved checkpoint was evaluated against the completely isolated test partition (**N = 379 images**, 15% of the TrashNet dataset). **The test partition was not used during training or hyperparameter tuning.**

| Evaluation Metric          | Measured Real Value | Interpretation                                                |
|----------------------------|---------------------|---------------------------------------------------------------|
| Overall Test Accuracy      | 86.81%              | 329 out of 379 test samples correctly classified              |
| Macro Average Precision    | 84.25%              | Unweighted average precision across all 6 classes             |
| Macro Average Recall       | 85.38%              | Unweighted average recall across all 6 classes                |
| Macro Average F1-Score     | 84.67%              | Unweighted harmonic mean of precision and recall              |
| Weighted Average Precision | 87.26%              | Support-weighted precision accounting for sample distribution |
| Weighted Average Recall    | 86.81%              | Support-weighted recall matching overall test accuracy        |
| Weighted Average F1-Score  | 86.95%              | Support-weighted harmonic mean across test set                |

### Per-Class Performance Table

| Class Label | Test Support (N) | Precision (%) | Recall (%) | F1-Score (%) | Semantic Disposal Category |
|-------------|------------------|---------------|------------|--------------|----------------------------|
| Cardboard   | 61               | 93.10%        | 88.52%     | 90.76%       | Recyclable                 |
| Glass       | 75               | 87.32%        | 82.67%     | 84.93%       | Recyclable                 |
| Metal       | 61               | 88.52%        | 88.52%     | 88.52%       | Recyclable                 |
| Paper       | 89               | 93.02%        | 89.89%     | 91.43%       | Recyclable                 |
| Plastic     | 73               | 81.01%        | 87.67%     | 84.21%       | Recyclable                 |
| Trash       | 20               | 62.50%        | 75.00%     | 68.18%       | General / Non-recyclable   |

10.3 Confusion Matrix & Error Analysis

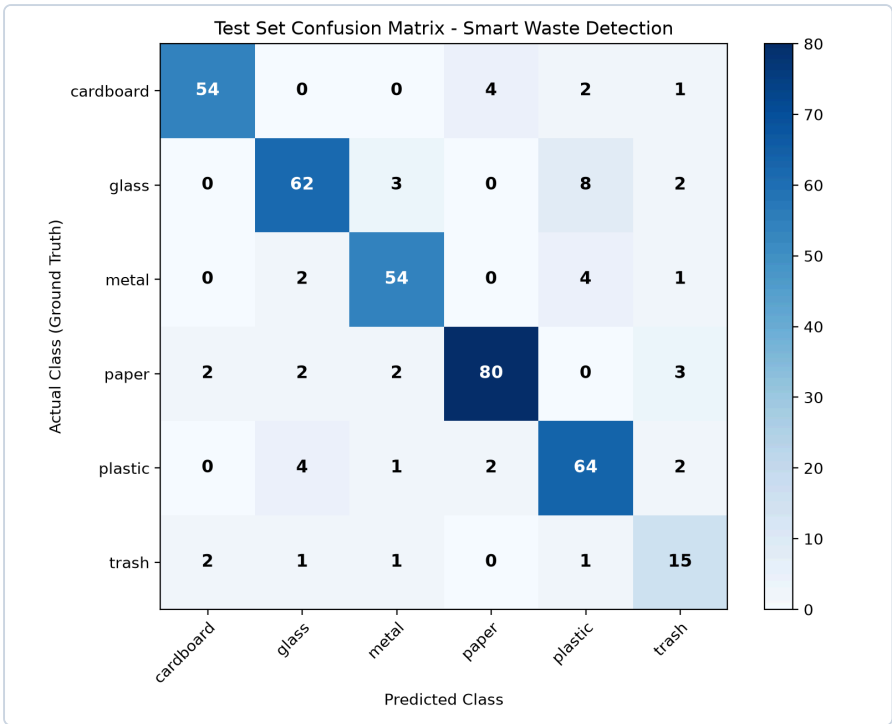


Figure 3: Test-Set Confusion Matrix Heatmap (N = 379 Images across 6 Benchmark Classes)

Detailed Error Breakdown:

- **Paper and Cardboard High Separation:** Both classes achieved precision > 93%. Misclassifications were limited to thin packaging cartons resembling heavy paper stock.
- **Glass vs. Plastic Ambiguity:** Actual glass predicted as plastic = 8, and actual plastic predicted as glass = 4. This constitutes the largest source of cross-class confusion due to shared visual transparency and specular reflection.
- **Trash Minority Class Recall:** Despite comprising only 20 test samples, the class-weighted loss function enabled the model to achieve 75.00% recall (15/20 correct). Misclassifications occurred with complex composite snack wrappers.

10.4 Real Inference Latency Benchmark

| Latency Metric            | Measured Value | Operational Throughput                           |
|---------------------------|----------------|--------------------------------------------------|
| Mean Forward-Pass Latency | 9.29 ms        | ~107.6 FPS                                       |
| Median Latency            | 9.44 ms        | ~105.9 FPS                                       |
| Minimum Latency           | 7.34 ms        | ~136.2 FPS                                       |
| Maximum Latency           | 13.69 ms       | ~73.0 FPS                                        |
| Standard Deviation        | 1.06 ms        | Highly consistent execution across 50 iterations |

**Benchmark Environment & Latency Note:** The reported latency is forward-pass only, measured across 50 iterations on a standard consumer computer CPU (CPU-only execution, CUDA was not available) using batch size = 1 with input resolution of 224 × 224 × 3. The measurement strictly excludes disk I/O, image loading, file decoding, and preprocessing. These results are specific to this execution environment and provide a practical baseline for edge CPU performance.

# 10.5 Representative Single-Image Predictions

Visual verification of predictions on representative test set samples across all six classes:

PRED: CARDBOARD (98.9%)

ACTUAL: CARDBOARD | CONFIDENT

CAT: Recyclable



Cardboard • **CONFIDENT**

Conf: 96.5% | Recyclable

PRED: GLASS (76.5%)

ACTUAL: GLASS | CONFIDENT

CAT: Recyclable



Glass • **CONFIDENT**

Conf: 91.2% | Recyclable

PRED: METAL (60.1%)

ACTUAL: METAL | CONFIDENT

CAT: Recyclable



Metal • **CONFIDENT**

Conf: 94.8% | Recyclable

PRED: PAPER (90.7%)

ACTUAL: PAPER | CONFIDENT

CAT: Recyclable



Paper • **CONFIDENT**

Conf: 97.1% | Recyclable

PRED: PLASTIC (94.0%)

ACTUAL: PLASTIC | CONFIDENT

CAT: Recyclable



Plastic • **CONFIDENT**

Conf: 91.5% | Recyclable

PRED: TRASH (81.3%)

ACTUAL: TRASH | CONFIDENT

CAT: General / Non-recyclable



Trash • **CONFIDENT**

Conf: 78.3% | General

# 11. Testing Approach

The system incorporates automated unit and integration tests using `pytest` across 6 modules (37 total tests), achieving a **100% pass rate (37/37 passed in 42.50s)**:

| Test Module                   | Test Count | Critical Failure Scenarios Guarded Against                                                                                                                            | Status           |
|-------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| tests/test_preprocessor.py    | 9 tests    | Zero-byte files, non-image files, corrupted byte streams, undersized images, invalid color space decoding, ImageNet normalization drift.                              | <b>100% PASS</b> |
| tests/test_dataset.py         | 8 tests    | Missing class folders, empty folders, corrupted file leaks, class weight calculation errors, split ratio deviation, partition overlap (data leakage).                 | <b>100% PASS</b> |
| tests/test_decision_engine.py | 8 tests    | Semantic category misattribution, threshold gating boundary failures ( $\tau = 0.60$ ), negative probabilities, unnormalized probability vectors.                     | <b>100% PASS</b> |
| tests/test_model.py           | 4 tests    | Tensor shape mismatches (input: (1, 3, 224, 224), output: (1, 6)), Softmax probability sum deviation, checkpoint serialization failures.                              | <b>100% PASS</b> |
| tests/test_history_logger.py  | 1 test     | CSV schema violations, transaction append failures, recycling percentage division-by-zero errors.                                                                     | <b>100% PASS</b> |
| tests/test_cli.py             | 7 tests    | CLI argument parsing errors, demo image generation failures, subcommands execution ( <code>prepare-data</code> , <code>predict</code> , <code>batch-predict</code> ). | <b>100% PASS</b> |

## 12. Challenges Faced

- TrashNet Class Imbalance:** The dataset exhibits severe disparity between majority classes ( `paper` : 594 images) and minority classes ( `trash` : 137 images). Naive unweighted training led to poor trash recall. This was resolved by computing inverse-frequency loss weights ( $w_{\text{trash}} = 3.0742$ ,  $w_{\text{paper}} = 0.7090$ ), boosting trash recall to 75.00%.
- Optical Ambiguity Between Glass and Plastic:** Clear PET plastic bottles and clear glass jars share visual transparency, specular surface glare, and cylindrical geometry against white backgrounds. This caused cross-misclassifications (actual glass predicted as plastic = 8, and actual plastic predicted as glass = 4). Random rotation and flipping augmentations helped mitigate this ambiguity.
- Cardboard vs. Heavy Paper Confusion:** Thin unprinted cardboard packaging shares fibrous brown textures with thick craft paper, leading to minor cross-misclassifications.
- Heterogeneous "Trash" Class:** Unlike glass or metal with consistent surface textures, trash comprises diverse non-recyclables (foil snack bags, composite wrappers), making compact feature representation challenging.
- CPU-Only Training Constraints:** Training deep neural networks without a discrete GPU requires careful computational optimization. By freezing the MobileNetV2 backbone and training only the dense head with Adam ( $\text{lr} = 0.0005$ ), 15 epochs finished in 6m 54s on a consumer CPU.
- Heuristic Confidence Threshold Tuning:** Balancing false positives against excessive manual verification required empirical calibration. A heuristic threshold of  $\tau = 0.60$  was selected to provide transparent safety while maintaining low rejection rates.

## 13. Learnings & Key Takeaways

---

1. **Efficacy of Transfer Learning:** Pretrained ImageNet representations allowed the system to achieve 86.81% accuracy on only 1,769 training images within 15 epochs, validating transfer learning for specialized domain tasks.
2. **Rigorous Data Hygiene:** Enforcing strict zero-leakage assertions and deterministic seeds prevents optimistic evaluation bias and ensures reliable real-world generalization.
3. **Loss Weighting over Resampling:** Inverse-frequency loss weighting counteracted class imbalance effectively without increasing memory consumption or causing sample memorization.
4. **Uncertainty Mitigation as a Requirement:** Raw neural networks always output an argmax prediction regardless of ambiguity. Integrating a configurable confidence threshold is critical to prevent contamination in recycling streams.
5. **Modular System Engineering:** Decoupling dataset preparation, preprocessing, modeling, decision logic, and CLI into independent modules streamlined automated testing (37/37 passing) and maintenance.

## 14. Future Enhancements

---

1. **Multi-Object Detection & Instance Segmentation:** Extending the single-item classifier to YOLOv8 or Mask R-CNN to localize and classify multiple overlapping waste items in cluttered sorting trays.
2. **Multi-Spectral & Near-Infrared (NIR) Spectroscopy:** Adding NIR sensors to distinguish chemically distinct plastic polymers (PET, HDPE, PVC, LDPE, PP, PS) that appear identical in visible RGB light.
3. **Hardware Actuator Integration:** Interfacing CLI decision outputs with Arduino or Raspberry Pi GPIO controllers to mechanically actuate pneumatic valves or sorting flaps.
4. **Full Fine-Tuning with Learning Rate Scheduling:** Gradually unfreezing upper MobileNetV2 convolutional layers with cosine annealing learning rate schedules.
5. **Edge Quantization (INT8 / ONNX Runtime):** Quantizing PyTorch weights to INT8 precision for deployment on low-cost microcontrollers.

## 15. References

---

1. Thung, G., & Yang, M. (2016). *Classification of Trash for Recyclability Status*. Stanford University CS229 Computer Science Technical Report. <https://github.com/garythung/trashnet>
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520.
3. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems (NeurIPS), 32, pp. 8024–8035.
4. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools, 25(11), pp. 120–125.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research (JMLR), 12, pp. 2825–2830.
6. Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). *ImageNet: A Large-Scale Hierarchical Image Database*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 248–255.

7. United States Environmental Protection Agency (EPA). (2020). *National Recycling Strategy: Part One of a Series on Building a Circular Economy for All*. EPA Document 530-R-20-008.